ITM SKILLS UNIVERSITY

B.Tech Computer Science and Engineering

Cohort: Demis Hassabis

DSA-II

Programming Assignment 1

Topic: Two-Pointer Technique

Removing Duplicates from a Sorted Array and Pair Sum in a Sorted Array

Student Name	Rashmeet Kaur Nanade
Roll Number	150096725165
College	ITM Skills University
Program	B.Tech Computer Science and Engineering
Cohort	Demis Hassabis
Subject	DSA-II
Academic Year	2026–27
Assignment	Programming Assignment 1
Topic	Two-Pointer Technique
Faculty	Prof. Dr. Aarti Pardeshi
GitHub Repository	https://github.com/krashmeet033-sys/DSA-2-Assignment-
Submission Date	31 October 2026

1. Introduction

The two-pointer technique uses two indices that move through a sorted array so that a problem is solved in a single pass instead of repeated scanning. This report solves Remove Duplicates (Part A) and Pair Sum (Part B), each with a simple reference version and an optimized two-pointer version, implemented in C++, Java and Python and compared on the required test cases.

2. Problem Interpretation

2.1 Part A: Remove Duplicates from a Sorted Array

A non-decreasing integer array is given. Rearrange it in place so that its first k positions hold each distinct value once, in the original order, and return k.

- **Input and output:** a sorted integer array (possibly empty, possibly with negative values) and the count k of distinct values.
- **In place:** the result is built inside the same array. Elements after index k − 1 are leftovers and are ignored.
- **Boundary cases:** an empty array returns 0 without reading any element, and a one-element array returns 1.
- **Two versions:** the reference version only counts the distinct values, while the optimized version also builds the unique prefix.

2.2 Part B: Pair Sum in a Sorted Array

A sorted integer array and a target T are given. Return indices (i, j) with i < j and arr[i] + arr[j] = T, or (-1, -1) if no such pair exists.

- **Distinct positions:** the same element cannot be used twice, but equal values at different positions are allowed, such as the two 2s in [1, 2, 2, 3].
- **Any valid pair:** if several pairs exist, any one is accepted. Each algorithm returns the first pair that its own search order reaches.
- **No solution:** if no pair has the required sum, or the array has fewer than two elements, the result is (-1, -1).

3. Algorithm Steps and Pseudocode

3.1 Part A, Approach 1: removeDuplicatesReference

The reference solution builds a separate list. Each element of the array is looked up in that list, and it is appended only when it is not already present. The size of the list at the end is the number of distinct values. The solution does not use the fact that the array is sorted, and it does not modify the input.

1. Create an empty list `result`.
2. For every index i of the array, set `found = false`.
3. Scan `result` from the start. If `arr[i]` equals some `result[j]`, set `found = true` and stop scanning.
4. If `found` is still false, append `arr[i]` to `result`.
5. After the last element, return the size of `result`.

```
FUNCTION removeDuplicatesReference(arr)
    result <- empty list
    FOR i <- 0 TO length(arr) - 1
        found <- false
        FOR j <- 0 TO length(result) - 1
            IF arr[i] = result[j] THEN
                found <- true
                BREAK
        IF found = false THEN append arr[i] to result
    RETURN length(result)
```

3.2 Part A, Approach 2: removeDuplicatesOptimized (slow and fast pointers)

The optimized solution keeps two indices. The slow pointer marks the position of the last distinct value that has been stored at the front of the array, so `arr[0..slow]` is always the unique prefix built so far. The fast pointer scans every remaining element exactly once. Because equal values are adjacent in a sorted array, a new distinct value is detected simply by comparing `arr[fast]` with `arr[slow]`.

Table 1: *Pointer behaviour in removeDuplicatesOptimized*

Aspect	Description
Initialization	If n = 0 the function returns 0 immediately, so no element is read. Otherwise slow = 0 (the first element is always unique) and fast starts at 1.
Loop condition	`for (fast = 1; fast < n; fast++)`. The loop runs while fast is a valid index, so it executes n − 1 times and never reads outside the array.
Comparison	`arr[fast] != arr[slow]`. If the two values differ, `arr[fast]` is a new distinct value. If they are equal, it repeats the last stored value.
Pointer update	fast increases by one in every iteration. slow increases by one only when a new value is found, and then `arr[slow] = arr[fast]` copies that value into the next free position of the prefix.
Termination	The loop ends when fast reaches n. Since slow is an index, the number of distinct values is `slow + 1`. A single-element array skips the loop and returns 1.

```
FUNCTION removeDuplicatesOptimized(arr)
    n <- length(arr)
    IF n = 0 THEN RETURN 0
    slow <- 0
    FOR fast <- 1 TO n - 1
        IF arr[fast] != arr[slow] THEN
            slow <- slow + 1
            arr[slow] <- arr[fast]
    RETURN slow + 1
```

3.3 Part B, Approach 1: pairSumBruteForce

The brute-force solution tests every possible pair of positions. The outer index i fixes the first element and the inner index j runs over all positions after i. Starting j at i + 1 guarantees that an element is never paired with itself and that each pair is tested only once. The first pair whose sum equals the target is returned, and (-1, -1) is returned if the loops finish without a match.

1. Set n to the length of the array.
2. For each i from 0 to n − 1, let j run from i + 1 to n − 1.
3. If `arr[i] + arr[j]` equals the target, return `(i, j)` at once.
4. If both loops end without a match, return `(-1, -1)`.

```
FUNCTION pairSumBruteForce(arr, target)
    n <- length(arr)
    FOR i <- 0 TO n - 1
        FOR j <- i + 1 TO n - 1
            IF arr[i] + arr[j] = target THEN RETURN (i, j)
    RETURN (-1, -1)
```

3.4 Part B, Approach 2: pairSumOptimized (opposite-end pointers)

The optimized solution places the left pointer on the smallest element and the right pointer on the largest element. In every iteration it computes `sum = arr[left] + arr[right]` and compares it with the target. The comparison tells exactly which pointer can be moved safely.

Table 2: *Pointer behaviour in pairSumOptimized*

Aspect	Description
Initialization	If the array has fewer than two elements, return (-1, -1) at once. Otherwise left = 0 and right = n − 1.
Loop condition	`while (left < right)`. The condition keeps the two pointers on different positions, so an element is never paired with itself.
Comparison	`sum = arr[left] + arr[right]` is compared with the target: equal, smaller or larger.
Pointer update	sum = target: return (left, right). sum < target: `left++`. sum > target: `right--`. Exactly one pointer moves by one step in each iteration.
Termination	The function stops as soon as a pair is found. Otherwise the pointers meet (left = right), every candidate has been ruled out, and (-1, -1) is returned.

Why left moves when the sum is too small. The right pointer holds the largest value still under consideration. If `arr[left] + arr[right]` is already smaller than the target, then `arr[left]` combined with any other remaining value, all of which are at most `arr[right]`, is smaller still. The element at `left` therefore cannot belong to any valid pair, and moving left one step to the right discards it safely. A larger left value is the only way to increase the sum.

Why right moves when the sum is too large. Symmetrically, the left pointer holds the smallest remaining value. If `arr[left] + arr[right]` already exceeds the target, then `arr[right]` combined with any other remaining value, all of which are at least `arr[left]`, is larger still. The element at `right` cannot be part of a valid pair, so right moves one step to the left to reduce the sum.

```
FUNCTION pairSumOptimized(arr, target)
    IF length(arr) < 2 THEN RETURN (-1, -1)
    left <- 0
    right <- length(arr) - 1
    WHILE left < right
        sum <- arr[left] + arr[right]
        IF sum = target THEN
            RETURN (left, right)
        ELSE IF sum < target THEN
            left <- left + 1
        ELSE
            right <- right - 1
    RETURN (-1, -1)
```

4. Implementation

The code below is taken from the submitted GitHub repository (`TwoPointerAssignment.cpp`, `.java` and `.py`): https://github.com/krashmeet033-sys/DSA-2-Assignment-. Each listing is copied from the source file without any change. In Java the methods are static members of the class `TwoPointerAssignment`.

4.1 File Header

The C++ and Java files start with the lines below; the Python file has no header. The closing brace of the Java class appears at the end of the driver listing in Section 4.6.

C++

```
#include <iostream>
#include <vector>
#include <utility>
using namespace std;
```

Java

```
import java.util.*;

public class TwoPointerAssignment {
```

4.2 Part A, Approach 1: removeDuplicatesReference

C++

```
// Reference approach: Uses a separate result vector
int removeDuplicatesReference(const vector<int>& arr) {
    vector<int> result;

    for (int i = 0; i < arr.size(); i++) {
        bool found = false;

        for (int j = 0; j < result.size(); j++) {
            if (arr[i] == result[j]) {
                found = true;
                break;
            }
```

```
        }

        if (!found) {
            result.push_back(arr[i]);
        }
    }

    return result.size();
}
```

Java

```
    // Reference approach: Uses a separate result list
    public static int removeDuplicatesReference(int[] arr) {
        ArrayList<Integer> result = new ArrayList<>();

        for (int i = 0; i < arr.length; i++) {
            boolean found = false;

            for (int j = 0; j < result.size(); j++) {
                if (arr[i] == result.get(j)) {
                    found = true;
                    break;
                }
            }

            if (!found) {
                result.add(arr[i]);
            }
        }

        return result.size();
    }
```

Python

```
# Reference approach: Uses a separate result list
def removeDuplicatesReference(arr):
    result = []

    for i in range(len(arr)):
        found = False

        for j in range(len(result)):
            if arr[i] == result[j]:
                found = True
                break

        if not found:
            result.append(arr[i])

    return len(result)
```

4.3 Part A, Approach 2: removeDuplicatesOptimized

C++

```
// Optimized approach: Slow and fast pointers
int removeDuplicatesOptimized(vector<int>& arr) {
    int n = arr.size();

    if (n == 0) {
        return 0;
    }

    int slow = 0;

    for (int fast = 1; fast < n; fast++) {
        if (arr[fast] != arr[slow]) {
            slow++;
            arr[slow] = arr[fast];
        }
    }

    return slow + 1;
}
```

Java

```
    // Optimized approach: Slow and fast pointers
    public static int removeDuplicatesOptimized(int[] arr) {
        int n = arr.length;

        if (n == 0) {
            return 0;
        }

        int slow = 0;

        for (int fast = 1; fast < n; fast++) {
            if (arr[fast] != arr[slow]) {
                slow++;
                arr[slow] = arr[fast];
            }
        }

        return slow + 1;
    }
```

Python

```
# Optimized approach: Slow and fast pointers
def removeDuplicatesOptimized(arr):
    n = len(arr)

    if n == 0:
        return 0
```

```
    slow = 0

    for fast in range(1, n):
        if arr[fast] != arr[slow]:
            slow += 1
            arr[slow] = arr[fast]

    return slow + 1
```

4.4 Part B, Approach 1: pairSumBruteForce

C++

```
// Reference approach: Brute force
pair<int, int> pairSumBruteForce(
        const vector<int>& arr, int target) {

    int n = arr.size();

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                return {i, j};
            }
        }
    }

    return {-1, -1};
}
```

Java

```
    // Reference approach: Brute force
    public static int[] pairSumBruteForce(int[] arr, int target) {
        int n = arr.length;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (arr[i] + arr[j] == target) {
                    return new int[]{i, j};
                }
            }
        }

        return new int[]{-1, -1};
    }
```

Python

```
# Reference approach: Brute force
def pairSumBruteForce(arr, target):
    n = len(arr)
```

```
    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] + arr[j] == target:
                return (i, j)

    return (-1, -1)
```

4.5 Part B, Approach 2: pairSumOptimized

C++

```
// Optimized approach: Opposite-end two pointers
pair<int, int> pairSumOptimized(
        const vector<int>& arr, int target) {

    if (arr.size() < 2) {
        return {-1, -1};
    }

    int left = 0;
    int right = arr.size() - 1;

    while (left < right) {
        int sum = arr[left] + arr[right];

        if (sum == target) {
            return {left, right};
        }
        else if (sum < target) {
            left++;
        }
        else {
            right--;
        }
    }

    return {-1, -1};
}
```

Java

```
    // Optimized approach: Opposite-end two pointers
    public static int[] pairSumOptimized(int[] arr, int target) {
        if (arr.length < 2) {
            return new int[]{-1, -1};
        }

        int left = 0;
        int right = arr.length - 1;

        while (left < right) {
            int sum = arr[left] + arr[right];
```

```
            if (sum == target) {
                return new int[]{left, right};
            }
            else if (sum < target) {
                left++;
            }
            else {
                right--;
            }
        }

        return new int[]{-1, -1};
    }
```

Python

```
# Optimized approach: Opposite-end two pointers
def pairSumOptimized(arr, target):
    if len(arr) < 2:
        return (-1, -1)

    left = 0
    right = len(arr) - 1

    while left < right:
        total = arr[left] + arr[right]

        if total == target:
            return (left, right)

        elif total < target:
            left += 1

        else:
            right -= 1

    return (-1, -1)
```

4.6 Display Helper and Main Driver

`displayArray` prints the first k elements of an array. The driver runs both versions of each part on the four required test arrays, plus the extra pair-sum validation test (array [1, 3, 5, 8] with target 6), and prints the results.

C++

```
// ====================================================
// FUNCTION TO DISPLAY ARRAY
// ====================================================

void displayArray(const vector<int>& arr, int k) {
    cout << "[";

    for (int i = 0; i < k; i++) {
```

```
        cout << arr[i];

        if (i < k - 1) {
            cout << ", ";
        }
    }

    cout << "]";
}

// ====================================================
// MAIN FUNCTION
// ====================================================

int main() {

    // PART A: REMOVE DUPLICATES

    cout << "PART A: REMOVE DUPLICATES" << endl;

    vector<vector<int>> duplicateTests = {
        {1, 1, 2, 2, 2, 3, 4, 4},
        {5},
        {},
        {-3, -3, -1, 0, 0}
    };

    for (int i = 0; i < duplicateTests.size(); i++) {
        vector<int> arr = duplicateTests[i];

        int referenceK = removeDuplicatesReference(arr);
        int k = removeDuplicatesOptimized(arr);

        cout << "\nTest Case " << i + 1 << endl;
        cout << "Reference Unique Count: "
             << referenceK << endl;

        cout << "Optimized Unique Count: "
             << k << endl;

        cout << "Unique Prefix: ";
        displayArray(arr, k);
        cout << endl;
    }

    // PART B: PAIR SUM

    cout << "\nPART B: PAIR SUM" << endl;

    vector<vector<int>> pairTests = {
        {1, 2, 4, 7, 11, 15},
        {1, 3, 5, 8},
        {2, 4, 6},
        {1, 2, 2, 3},
        {1, 3, 5, 8}  // Additional corrected test
    };
```

```
    vector<int> targets = {15, 7, 20, 4, 6};

    for (int i = 0; i < pairTests.size(); i++) {
        vector<int> arr = pairTests[i];
        int target = targets[i];

        pair<int, int> brute =
            pairSumBruteForce(arr, target);

        pair<int, int> optimized =
            pairSumOptimized(arr, target);

        cout << "\nTest Case " << i + 1 << endl;
        cout << "Array: ";
        displayArray(arr, arr.size());
        cout << endl;

        cout << "Target: " << target << endl;

        cout << "Brute Force Indices: ("
             << brute.first << ", "
             << brute.second << ")" << endl;

        cout << "Optimized Indices: ("
             << optimized.first << ", "
             << optimized.second << ")" << endl;

        if (optimized.first != -1) {
            cout << "Pair Values: "
                 << arr[optimized.first] << " + "
                 << arr[optimized.second] << " = "
                 << target << endl;
        }
    }

    return 0;
}
```

Java

```
    // ====================================================
    // FUNCTION TO DISPLAY ARRAY
    // ====================================================

    public static void displayArray(int[] arr, int k) {
        System.out.print("[");

        for (int i = 0; i < k; i++) {
            System.out.print(arr[i]);

            if (i < k - 1) {
                System.out.print(", ");
            }
        }
```

```
        System.out.print("]");
    }

    // ========================================================
    // MAIN FUNCTION
    // ========================================================

    public static void main(String[] args) {

        // PART A: REMOVE DUPLICATES

        System.out.println("PART A: REMOVE DUPLICATES");

        int[][] duplicateTests = {
            {1, 1, 2, 2, 2, 3, 4, 4},
            {5},
            {},
            {-3, -3, -1, 0, 0}
        };

        for (int i = 0; i < duplicateTests.length; i++) {
            int[] arr = duplicateTests[i].clone();

            int referenceK = removeDuplicatesReference(arr);
            int k = removeDuplicatesOptimized(arr);

            System.out.println("\nTest Case " + (i + 1));
            System.out.println("Reference Unique Count: " + referenceK);
            System.out.println("Optimized Unique Count: " + k);
            System.out.print("Unique Prefix: ");
            displayArray(arr, k);
            System.out.println();
        }

        // PART B: PAIR SUM

        System.out.println("\nPART B: PAIR SUM");

        int[][] pairTests = {
            {1, 2, 4, 7, 11, 15},
            {1, 3, 5, 8},
            {2, 4, 6},
            {1, 2, 2, 3},
            {1, 3, 5, 8}  // Additional corrected test
        };

        int[] targets = {15, 7, 20, 4, 6};

        for (int i = 0; i < pairTests.length; i++) {
            int[] arr = pairTests[i];
            int target = targets[i];

            int[] brute = pairSumBruteForce(arr, target);
            int[] optimized = pairSumOptimized(arr, target);

            System.out.println("\nTest Case " + (i + 1));
            System.out.println("Array: " + Arrays.toString(arr));
```

```
            System.out.println("Target: " + target);

            System.out.println("Brute Force Indices: ("
                    + brute[0] + ", " + brute[1] + ")");

            System.out.println("Optimized Indices: ("
                    + optimized[0] + ", " + optimized[1] + ")");

            if (optimized[0] != -1) {
                System.out.println("Pair Values: "
                        + arr[optimized[0]] + " + "
                        + arr[optimized[1]] + " = "
                        + target);
            }
        }
    }
}
```

Python

```
# ======================================================
# FUNCTION TO DISPLAY ARRAY
# ======================================================

def displayArray(arr, k):
    print(arr[:k])


# ======================================================
# MAIN FUNCTION
# ======================================================

if __name__ == "__main__":

    # PART A: REMOVE DUPLICATES

    print("PART A: REMOVE DUPLICATES")

    duplicateTests = [
        [1, 1, 2, 2, 2, 3, 4, 4],
        [5],
        [],
        [-3, -3, -1, 0, 0]
    ]

    for i in range(len(duplicateTests)):
        arr = duplicateTests[i].copy()

        referenceK = removeDuplicatesReference(arr)
        k = removeDuplicatesOptimized(arr)

        print("\nTest Case", i + 1)
        print("Reference Unique Count:", referenceK)
        print("Optimized Unique Count:", k)
        print("Unique Prefix:", end=" ")
```

```
        displayArray(arr, k)

    # PART B: PAIR SUM

    print("\nPART B: PAIR SUM")

    pairTests = [
        [1, 2, 4, 7, 11, 15],
        [1, 3, 5, 8],
        [2, 4, 6],
        [1, 2, 2, 3],
        [1, 3, 5, 8]  # Additional corrected test
    ]

    targets = [15, 7, 20, 4, 6]

    for i in range(len(pairTests)):
        arr = pairTests[i]
        target = targets[i]

        brute = pairSumBruteForce(arr, target)
        optimized = pairSumOptimized(arr, target)

        print("\nTest Case", i + 1)
        print("Array:", arr)
        print("Target:", target)
        print("Brute Force Indices:", brute)
        print("Optimized Indices:", optimized)

        if optimized != (-1, -1):
            left, right = optimized
            print("Pair Values:",
                  arr[left], "+", arr[right],
                  "=", target)
```

5. Dry Runs

5.1 Dry Run 1: removeDuplicatesOptimized

Input: [1, 1, 2, 2, 2, 3, 4, 4], so n = 8. Expected result: k = 4 and unique prefix [1, 2, 3, 4]. The slow value shown in each row is the value at the start of that iteration, before any update. The last column shows the whole array after the iteration.

Table 3: *Dry run of removeDuplicatesOptimized on [1, 1, 2, 2, 2, 3, 4, 4]*

Step	fast	slow	arr[fast]	arr[slow]	Comparison	Action	Updated array state
Init	-	0	-	1	n = 8, so n != 0	slow = 0; the loop will start at fast = 1	[1, 1, 2, 2, 2, 3, 4, 4]
1	1	0	1	1	1 != 1 is false	Equal: duplicate, skip it	[1, 1, 2, 2, 2, 3, 4, 4]
2	2	0	2	1	2 != 1 is true	Different: slow becomes 1; arr[1] = 2	[1, 2, 2, 2, 2, 3, 4, 4]
3	3	1	2	2	2 != 2 is false	Equal: duplicate, skip it	[1, 2, 2, 2, 2, 3, 4, 4]
4	4	1	2	2	2 != 2 is false	Equal: duplicate, skip it	[1, 2, 2, 2, 2, 3, 4, 4]
5	5	1	3	2	3 != 2 is true	Different: slow becomes 2; arr[2] = 3	[1, 2, 3, 2, 2, 3, 4, 4]
6	6	2	4	3	4 != 3 is true	Different: slow becomes 3; arr[3] = 4	[1, 2, 3, 4, 2, 3, 4, 4]
7	7	3	4	4	4 != 4 is false	Equal: duplicate, skip it	[1, 2, 3, 4, 2, 3, 4, 4]
End	8	3	-	-	fast < n is false (8 < 8)	Loop ends; return slow + 1 = 4	[1, 2, 3, 4, 2, 3, 4, 4]

Result: the function returns k = slow + 1 = 4. The first 4 positions of the array hold the unique prefix [1, 2, 3, 4], exactly as expected. The remaining positions [2, 3, 4, 4] are leftovers from the earlier contents of the array and are ignored, which is allowed by the problem statement. Note that fast examined each of the seven remaining elements exactly once, and slow moved only three times, once for each new distinct value (2, 3 and 4).

5.2 Dry Run 2: pairSumOptimized

Input: [1, 2, 4, 7, 11, 15] with target = 15. Expected result: indices (2, 4), values 4 and 11. Initially left = 0 and right = 5.

Table 4: *Dry run of pairSumOptimized on [1, 2, 4, 7, 11, 15] with target 15*

Iteration	left	right	arr[left]	arr[right]	Sum	Comparison with target (15)	Pointer movement
1	0	5	1	15	16	16 > 15 (too large)	right-- : right becomes 4
2	0	4	1	11	12	12 < 15 (too small)	left++ : left becomes 1
3	1	4	2	11	13	13 < 15 (too small)	left++ : left becomes 2
4	2	4	4	11	15	15 = 15 (match)	Stop: return (2, 4)

Result: the final pair is indices (2, 4) with values 4 + 11 = 15. The two-pointer search needed only four iterations. The brute-force version tests the pairs in the order (0,1), (0,2), … and reaches (2, 4) only after 11 sums have been computed, because pairs with i = 0 and i = 1 are all tried first. Both functions return the same answer, as the terminal output in Section 11 confirms.

6. Test Cases and Results

The inputs below are the test arrays hard-coded in the source files. The actual outputs are the values printed by the programs and shown in the terminal captures of Section 11; all three languages produced identical results. In the tables, "ref" stands for the reference or brute-force version and "opt" for the optimized version.

6.1 Part A: Remove Duplicates

Table 5: *Test cases for duplicate removal*

Test	Category	Input array	Expected k and prefix	Actual k (ref / opt)	Actual unique prefix	Status
1	Duplicate-heavy	[1, 1, 2, 2, 2, 3, 4, 4]	k = 4 [1, 2, 3, 4]	4 / 4	[1, 2, 3, 4]	PASS
2	Single element	[5]	k = 1 [5]	1 / 1	[5]	PASS
3	Empty array	[ ]	k = 0 no invalid access	0 / 0	[ ]	PASS
4	Negatives and zero	[-3, -3, -1, 0, 0]	k = 3 [-3, -1, 0]	3 / 3	[-3, -1, 0]	PASS

In all four tests the reference count and the optimized count agree with the expected value of k, and the unique prefix produced by the optimized function matches the expected prefix. The empty array returns 0 without any element access, because the guard `n == 0` runs before the loop.

6.2 Part B: Pair Sum

Table 6: *Test cases for pair sum (Test 5 is an extra validation test)*

Test	Input array	Target	Expected (assignment sheet)	Correct expected result	Actual: brute force	Actual: optimized	Status
1	[1, 2, 4, 7, 11, 15]	15	(2, 4)	(2, 4)	(2, 4)	(2, 4)	PASS
2	[1, 3, 5, 8]	7	(0, 2)	(-1, -1)	(-1, -1)	(-1, -1)	PASS †
3	[2, 4, 6]	20	(-1, -1)	(-1, -1)	(-1, -1)	(-1, -1)	PASS
4	[1, 2, 2, 3]	4	Any valid pair	(0, 3) or (1, 2)	(0, 3)	(0, 3)	PASS ‡
5 (extra)	[1, 3, 5, 8]	6	Not in sheet	(0, 2)	(0, 2)	(0, 2)	PASS

6.3 Discussion of Test 2, Test 4 and the Extra Validation Test

† Test 2: array [1, 3, 5, 8], target 7. The assignment sheet lists (0, 2) as the expected pair. Those indices refer to the values 1 and 5, whose sum is 6 and not 7, so the stated expectation appears to contain a small slip. This can be checked by listing every pair of the array:

Table 7: *All pair sums of [1, 3, 5, 8]*

Pair (indices)	(0, 1)	(0, 2)	(0, 3)	(1, 2)	(1, 3) and (2, 3)
Values	1 + 3	1 + 5	1 + 8	3 + 5	3 + 8 and 5 + 8
Sum	4	6	9	8	11 and 13

None of the six sums equals 7, so no valid pair exists and the mathematically correct answer is (-1, -1). Both implementations return exactly this value. The programs were therefore left unchanged and their output was

not adjusted to match the printed expectation; the test is marked PASS because the actual output is the correct answer. The pointer trace for this input is given in Table 8 below: the pointers meet at left = right = 1 without ever hitting a sum of 7.

‡ Test 4: array [1, 2, 2, 3], target 4. Two valid pairs exist: (0, 3), since 1 + 3 = 4, and (1, 2), since 2 + 2 = 4. The second pair uses the two equal values at different positions, which is permitted. The problem accepts any valid pair of distinct indices, so (0, 3) is a correct answer. Both functions return it because it is the first pair that their search orders reach: the brute-force loops meet it at i = 0, j = 3, and the two-pointer search meets it in its very first iteration, where left = 0 and right = 3.

Test 5, extra validation test: array [1, 3, 5, 8], target 6. This test is not part of the assignment sheet. It was added to the programs to check the case that Test 2 appears to have intended, namely a successful search on the same array. Here 1 + 5 = 6, so the pair (0, 2) is both the expected and the actual result in both versions, and the printed values are 1 + 5 = 6.

Table 8: *Pointer traces for the edge cases of Part B (optimized version)*

Case	Iteration	left	right	Sum	Compare	Movement / result
[1, 3, 5, 8], T = 7	1	0	3	9	9 > 7	right-- : right becomes 2
	2	0	2	6	6 < 7	left++ : left becomes 1
	3	1	2	8	8 > 7	right-- : right becomes 1
	End	1	1	-	left < right is false	Loop ends; return (-1, -1)
[1, 3, 5, 8], T = 6	1	0	3	9	9 > 6	right-- : right becomes 2
	2	0	2	6	6 = 6	Stop: return (0, 2)
[1, 2, 2, 3], T = 4	1	0	3	4	4 = 4	Stop: return (0, 3)

Taken together, the eight required tests and the extra validation test cover normal inputs, duplicate-heavy input, an empty array, a single element, negative values, a no-solution case caused by a target that is too large, a no-solution case in which the pointers cross all possible sums, and a case with equal values at different positions.

7. Complexity Analysis

Let n be the length of the array and k the number of distinct values it contains, so that k ≤ n. Auxiliary space means the extra memory used apart from the input array itself.

7.1 Before and After Optimization

Table 9: *Reference and brute-force versions compared with the optimized versions*

Problem	Function (version)	Time	Auxiliary space	Why this complexity applies
Remove Duplicates	removeDuplicatesReference (before)	$O(n^2)$	O(k) worst case O(n)	Each of the n elements is searched for in `result`, which can hold up to k values, so up to n × k comparisons are made; with few duplicates this approaches n(n − 1)/2. `result` keeps one copy of every distinct value, which is O(n) when there are no duplicates.
Remove Duplicates	removeDuplicatesOptimized (after)	O(n)	O(1)	One loop runs n − 1 times, and each pass does one comparison and at most one write. Only n, `slow` and `fast` are stored, because the array itself is overwritten.
Pair Sum	pairSumBruteForce (before)	$O(n^2)$	O(1)	The nested loops test all n(n − 1)/2 pairs when no pair matches. Only `i`, `j` and n are stored.
Pair Sum	pairSumOptimized (after)	O(n)	O(1)	Every iteration moves one pointer one step inward, so `right − left` falls from n − 1 to 0 in at most n − 1 iterations. Only `left`, `right` and `sum` are stored.

In both parts the optimized version improves the running time from quadratic to linear, and for duplicate removal it also reduces the auxiliary space from O(k), up to O(n), to a constant, because the distinct values are written back into the array that is being scanned. For pair sum the brute-force version already uses constant space, so the gain there is purely in time. To put the difference in perspective, an array of 100,000 elements may need about five billion pair checks in the brute-force search, but at most about 100,000 pointer moves in the two-pointer search. Section 8 explains why the sorted order makes this possible.

7.2 Comparison across C++, Java and Python

Table 10: *Time and auxiliary space of every algorithm in each language*

Algorithm	C++	Java	Python	Language-specific reason
Remove Duplicates: result list	$O(n^2)$ O(k)	$O(n^2)$ O(k)	$O(n^2)$ O(k)	`result` is a `vector` in C++, an `ArrayList` in Java and a `list` in Python. Reading by index and appending cost O(1) in all three, so the nested scan decides the time.
Remove Duplicates: slow and fast pointers	O(n) O(1)	O(n) O(1)	O(n) O(1)	The function works directly on the `vector`, the `int[]` or the list; reading and assigning an element is O(1) in all three languages.
Pair Sum: brute force	$O(n^2)$ O(1)	$O(n^2)$ O(1)	$O(n^2)$ O(1)	The nested index loops are the same everywhere. The answer is a fixed-size pair (`pair`, `int[]` or tuple), which is constant space.
Pair Sum: two pointers	O(n) O(1)	O(n) O(1)	O(n) O(1)	The single loop performs one addition, one comparison and one pointer move per step in every language.

In each cell the first line is the time complexity and the second is the auxiliary space. Big-O depends on the algorithm and not on the language, so all three implementations have identical complexity. The languages differ only in constant factors (Python, for example, is interpreted), and those were not measured in this assignment. The copies of the test arrays made by the drivers are part of the test harness and not of the algorithms.

8. How Sorted Order Enables Optimization

Both optimized algorithms are correct only because the input is sorted. Without that guarantee, the reference-style solutions would still work, but the pointer-based ones would not.

8.1 Duplicate removal

In a sorted array all copies of a value form one contiguous block. When the fast pointer meets a value, the only question is whether it is the same as the block that has just been stored, and that value is exactly `arr[slow]`. A single comparison with one element therefore replaces the search through the whole result list that the reference solution performs. Once a block is finished, it can never appear again later in the array, so earlier values never have to be remembered.

The dependence on sorted order can be seen with the unsorted array [1, 2, 1]. The optimized function would store 1, then 2, and then compare the last 1 with `arr[slow] = 2`, find them different and count 1 a second time, returning 3 instead of 2. The reference solution would still return the correct answer 2 for this array because it checks the entire list.

8.2 Pair sum

The pointer movements in Part B rest on a simple invariant: every valid pair (i, j) with i < j satisfies left ≤ i and j ≤ right. At the start this is true because left = 0 and right = n − 1. If the current sum is smaller than the target, then for any j ≤ right the sorted order gives `arr[left] + arr[j] ≤ arr[left] + arr[right] < target`, so no valid pair can contain `left`; moving left to the right keeps the invariant. If the current sum is larger than the target, then for any i ≥ left we have `arr[i] + arr[right] ≥ arr[left] + arr[right] > target`, so no valid pair can contain `right`; moving right to the left also keeps the invariant.

Each step therefore removes one element from the search for good, and when the pointers meet, the invariant shows that no valid pair is left. In the brute-force version the order of the elements is never used, and every pair is examined even when its sum is obviously too small or too large. Sorting allows one comparison to rule out an entire row of pairs at once, which is where the reduction from $O(n^2)$ to $O(n)$ comes from.

9. Compilation and Execution Instructions

All three programs are stored in the same folder under the common name `TwoPointerAssignment`. They need no input from the keyboard, because the test arrays are defined inside the source code. The commands below are run from a terminal opened in that folder. They require a C++ compiler that supports C++17 (for example g++), a Java Development Kit (for `javac` and `java`), and Python 3.

Table 11: *Commands for compiling and running each program*

Language	Compile command	Run command	Output file
C++	g++ -std=c++17 TwoPointerAssignment.cpp -o TwoPointerAssignment	./TwoPointerAssignment	TwoPointerAssignment (executable)
Java	javac TwoPointerAssignment.java	java TwoPointerAssignment	TwoPointerAssignment.class
Python	Not required (interpreted)	python3 TwoPointerAssignment.py	None

- The Java source must be saved as `TwoPointerAssignment.java`, because the public class has the same name. The compilation creates `TwoPointerAssignment.class`, which is the file that `java` then runs.
- On Windows, the C++ executable is started with `TwoPointerAssignment.exe` (or `.\TwoPointerAssignment`), and Python is usually started with `python` or `py` instead of `python3`.
- A successful compilation prints nothing in either C++ or Java. Any message from the compiler indicates an error that must be fixed before the program can be run.
- The source files, the compiled `TwoPointerAssignment.class` and the C++ executable are also available in the GitHub repository whose link is given on the cover page.

10. Sample Input and Output

The programs print the same text in all three languages, and the index pairs print as (i, j) in every one of them. The table gives representative samples taken from the program output. The complete output of each run, covering all four Part A tests and all five Part B tests, is shown in the terminal captures of Section 11.

Table 12: *Representative sample inputs and outputs*

Part	Input	Output
A: duplicates	[1, 1, 2, 2, 2, 3, 4, 4]	Reference Unique Count: 4 Optimized Unique Count: 4 Unique Prefix: [1, 2, 3, 4]
A: empty array	[ ]	Reference Unique Count: 0 Optimized Unique Count: 0 Unique Prefix: []
B: pair exists	[1, 2, 4, 7, 11, 15], target 15	Brute Force Indices: (2, 4) Optimized Indices: (2, 4) Pair Values: 4 + 11 = 15
B: no pair	[2, 4, 6], target 20	Brute Force Indices: (-1, -1) Optimized Indices: (-1, -1)

11. Execution Evidence

The following screenshots were captured from the terminal of Visual Studio Code while each program was compiled and run. Because the output is longer than the terminal window, each run is shown in two captures: the first shows the commands and the beginning of the output, and the second shows the end of the output. For all three languages, the compilation finished without any error message and the programs ran to completion.

11.1 C++ Execution

Command used: `g++ -std=c++17 TwoPointerAssignment.cpp -o TwoPointerAssignment`, followed by `./TwoPointerAssignment`.

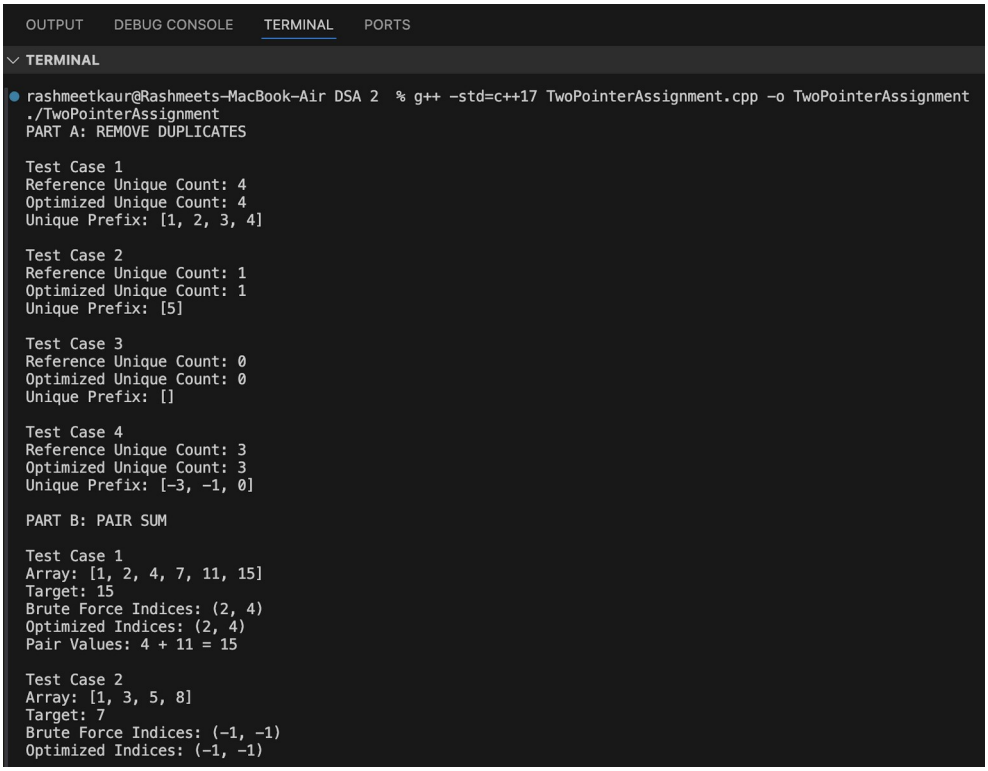


Figure 1: *C++ run: compilation commands and the first part of the output*

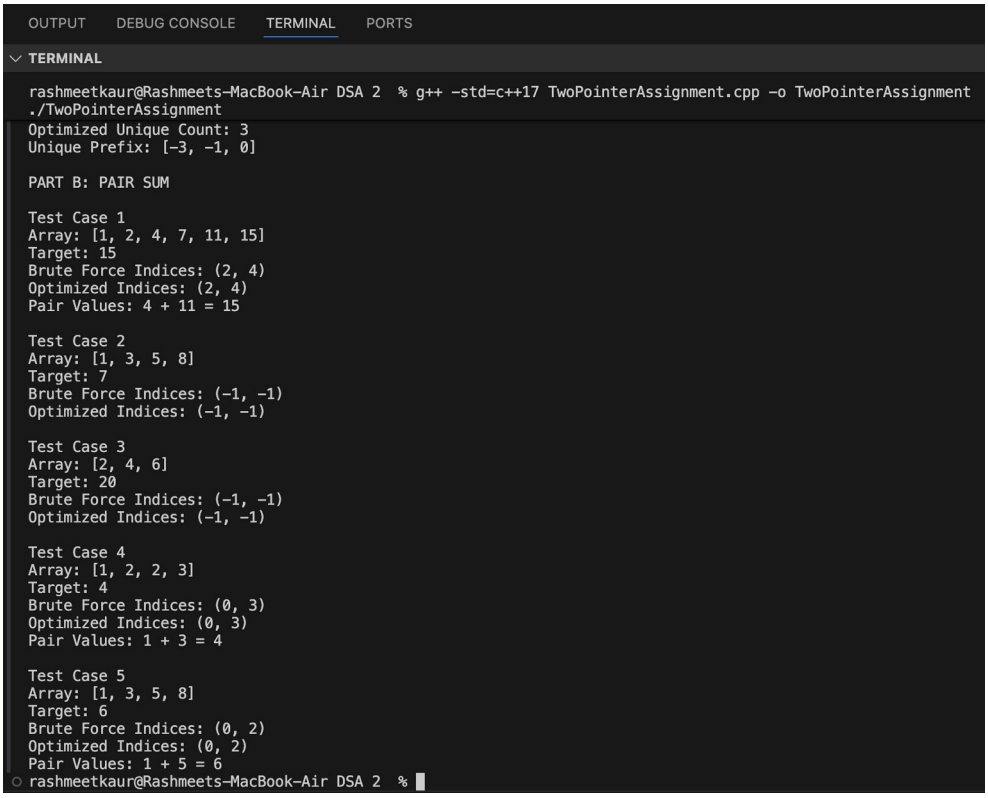


Figure 2: *C++ run: the end of the output, including the extra validation test*

The C++ program compiled under the C++17 standard and ran successfully. Part A reports the counts 4, 1, 0 and 3, and Part B reports identical indices for the brute-force and the optimized functions in all five tests.

11.2 Java Execution

Commands used: `javac TwoPointerAssignment.java`, followed by `java TwoPointerAssignment`.

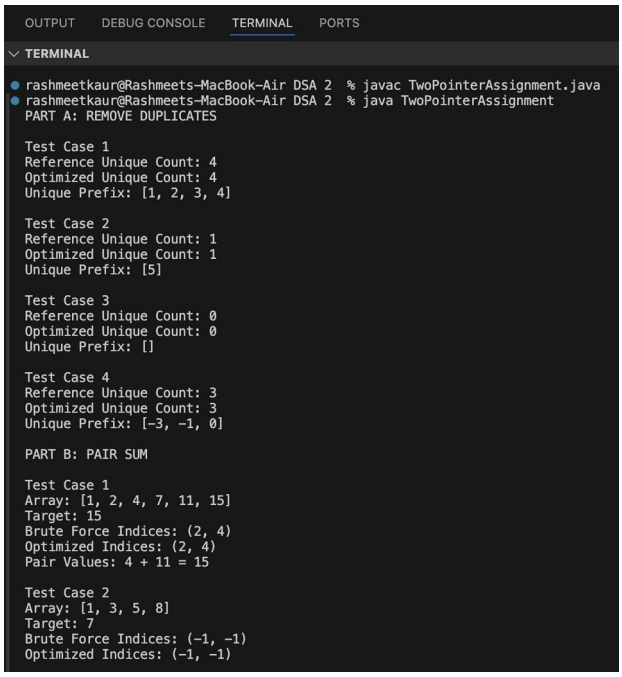


Figure 3: *Java run: compilation, run command and the first part of the output*

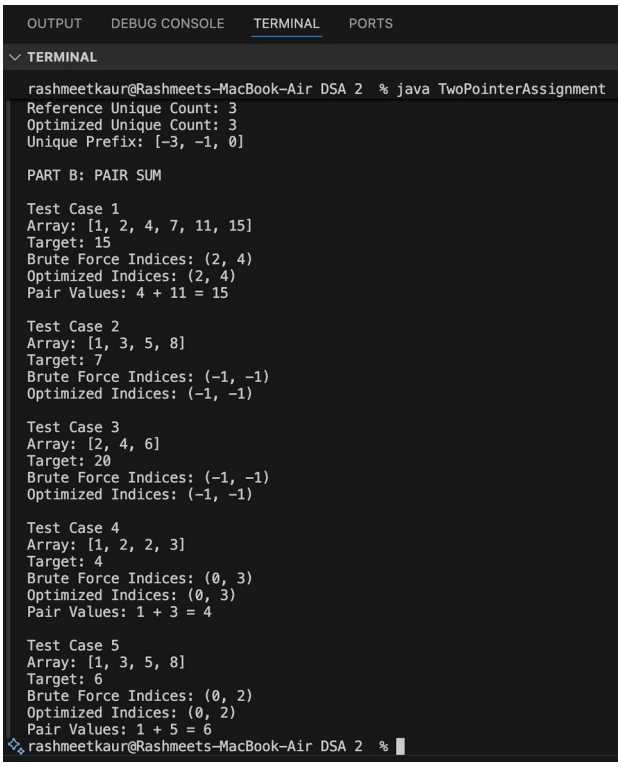


Figure 4: *Java run: the end of the output*

The Java source was compiled by `javac` without any message and the class was run with `java`. The output is identical, line for line, to the C++ output.

11.3 Python Execution

Command used: `python3 TwoPointerAssignment.py`.

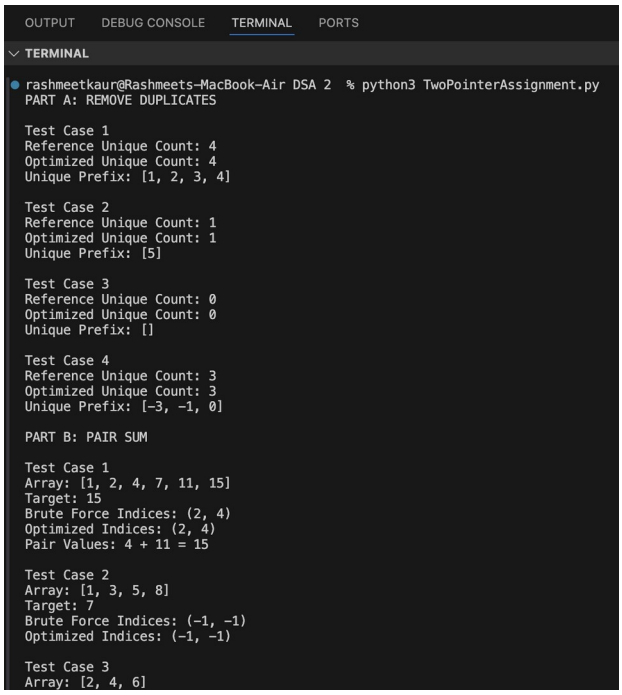


Figure 5: *Python run: command and the first part of the output*

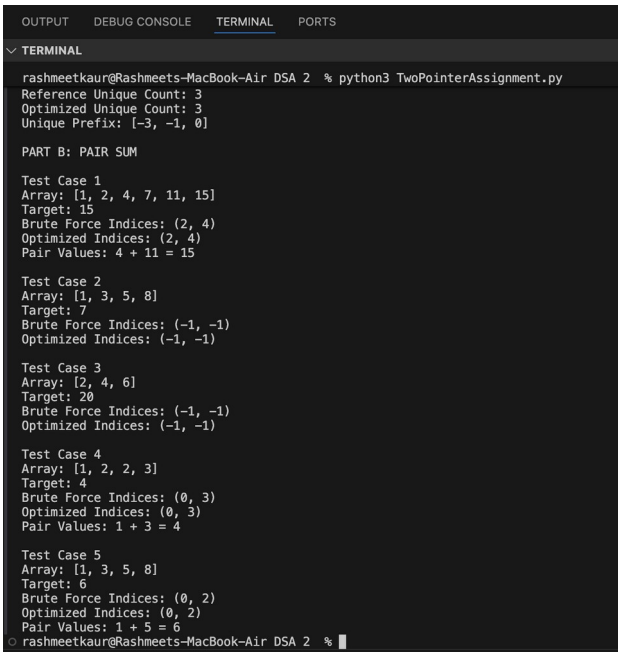


Figure 6: *Python run: the end of the output*

The Python program ran without errors and printed the same results as the C++ and Java programs. All four duplicate-removal tests gave the expected count and prefix, and the five pair-sum tests gave the indices listed in Table 6.

12. Conclusion

This assignment solved two problems on sorted arrays in two ways each. The reference and brute-force solutions are easy to write and correct for any input, but they take quadratic time. The two-pointer solutions use the sorted order to avoid repeated work: the slow and fast pointers remove duplicates in place in one pass with O(1) extra space, and the left and right pointers find a pair with a given sum in a single inward sweep. Section 8 showed that the pointer movements are justified by the ordering, which is why the optimized algorithms are correct and not just fast.

All four functions were implemented identically in C++17, Java and Python and run on the required test arrays. The reference and optimized versions agreed on every test, and the three languages produced the same output. The only difference from the assignment sheet is for the test with array [1, 3, 5, 8] and target 7: the sheet lists (0, 2), but 1 + 5 = 6, so no pair sums to 7 and the correct result (-1, -1) was returned by both versions. The extra validation test with target 6 returns (0, 2) as expected, and the test with [1, 2, 2, 3] and target 4 returns the valid pair (0, 3).

Working through the dry runs and the correctness invariant gave a clear understanding of when a pointer may be moved and why. The same idea of letting an ordering decide which candidates can be discarded appears in many other array and string problems, which makes the two-pointer technique a useful general tool.